

Materia:	Programación I		
Nivel:	1º Cuatrimestre		
Tipo de Examen:	Segundo Parcial		
Apellido ⁽¹⁾ :		Fecha:	
Nombre/s ⁽¹⁾ :		Docente a cargo ⁽²⁾ :	
División ⁽¹⁾ :		Nota ⁽²⁾ :	
DNI ⁽¹⁾ :		Firma ⁽²⁾ :	

(1) Campos a completar solo por el estudiante en caso de imprimir este enunciado en papel.

(2) Campos a completar solo por el docente en caso de imprimir este enunciado en papel.

Objetivos de Aprobación No Directa (Calificación de 4 a 5 puntos):

Que el estudiante:

- 1) Aplique técnicas de programación que involucre el adecuado manejo de listas, diccionarios, sets y tuplas.
- 2) Logre manipular de manera eficiente strings y sus métodos.
- 3) Logre leer y escribir archivos de distintos tipos, **con las técnicas vistas en clase.**
- 4) Aplique de manera eficiente funciones, teniendo en cuenta el paradigma funcional y los principios DRY (Don't Repeat Yourself).
- 5) Logre desarrollar el juego en consola, haciendo uso de las distintas técnicas de programación provistas en la cátedra.

Nota: para resolver la lógica del juego, podrán aplicar cualquier concepto visto en la primera parte de la materia

Objetivos de Aprobación Directa (Calificación de 6 a 10 puntos):

Que el estudiante:

- 1) Aplique técnicas de programación relacionadas con la biblioteca Pygame:
 - Configuraciones
 - Posicionamiento
 - Manipulación de imágenes
 - Movimientos
 - Sonidos
 - Eventos
 - Colisiones
- 2) Logre desarrollar un juego en pygame estéticamente prolijo y funcional, sin descuidar los objetivos establecidos para la aprobación no directa.

Juego de Preguntas y Respuestas Multietapa

Objetivo general:

Desarrollar un juego de preguntas y respuestas, primero en consola y luego en entorno gráfico (Pygame), que permita poner en práctica estructuras de datos complejas, archivos externos, modularidad, programación funcional y diseño accesible orientado a distintas neurodivergencias.

Deberán pensar en la temática del juego y la estructura del mismo (serpientes y escaleras, programa de tv de preguntas y respuestas, juego de la oca, preguntados, pasapalabra, etc)

Presentar un pequeño boceto de la idea del juego (jugabilidad en consola: cómo se juega) y luego cómo se vería con una interfaz gráfica (que se implementará en pygame). De esta manera se definirá el alcance del juego.

Sprints de desarrollo

◆ Sprint 1 – Versión básica en consola

Desarrollar el núcleo del juego en consola. El juego debe:

- Contener al menos 7 preguntas en total, organizadas en una **lista de diccionarios**.
- Cada pregunta tendrá una categoría (las categorías tienen que tener relación con la temática que eligieron del juego, por ejemplo superhéroes) y dificultad (otorga más o menos puntaje).
- Mostrar el enunciado, las opciones y registrar respuestas.
- Calcular y mostrar el puntaje final.

Se evaluará:

- Uso de estructuras de datos (listas, diccionarios, sets, tuplas).
- Control de flujo y condicionales.

◆ Sprint 2 – Archivos externos, configuración y modularidad

Separar la lógica del juego y cargar la información desde archivos externos:

- Las **preguntas** deben leerse desde un CSV. El dato deberá estar compuesto por: pregunta, opciones, respuesta correcta, categoría, dificultad y puntaje (en caso de necesitar más atributos, puede agregarlos)
- La **configuración** del juego se cargará desde un archivo JSON:
 - Cantidad de preguntas del juego (niveles).
 - Tiempo.
 - Accesibilidad:
 - Neurotípico (por defecto)
 - Neurodivergente (se setea desde el juego según adaptación que elijan)

- Las **estadísticas** se guardarán en un archivo CSV o JSON.
- El código debe estar modularizado, con funciones reutilizables y funcionales.
- Realizar las validaciones pertinentes a la hora de leer/escribir en archivos.

Se evaluará:

- Lectura/escritura de archivos.
- Modularización.

♦ **Sprint 3 – Estadísticas y modos de juego**

Mejorar la jugabilidad y agregar análisis del desempeño del jugador:

- Mostrar estadísticas de aciertos, errores, tiempo promedio, etc.
- Permitir jugar con varios perfiles y turnos.
- Posibilidad de reintentar preguntas incorrectas (si es necesario).

Se evaluará:

- Uso de tuplas y sets.
- Funciones puras y funciones genéricas y reutilizables.
- Persistencia y análisis de datos.

♦ **Sprint 4 – Juego completo en consola con configuraciones y accesibilidad**

Objetivo:

Consolidar un juego 100% funcional en consola, incluyendo perfiles de jugador, estadísticas, y las opciones de accesibilidad leídas desde un archivo JSON.

Requisitos nuevos:

- Incluir **resumen final y estadísticas**: porcentaje de aciertos, preguntas más falladas, tiempo promedio (si aplica).
- Incluir al menos **dos perfiles de jugador** con puntajes diferenciados (crear un archivo para guardar los datos de los jugadores, para luego poder loguearse en el juego y obtener el historial de estadísticas)

Se evaluará:

- Integración completa de funciones, archivos, estructuras.
- Coherencia entre el archivo de configuración y el comportamiento del juego.
- Uso de sets, tuplas y funciones genéricas y reutilizables para cálculo estadístico.

♦ **Sprint 5 – Versión gráfica con Pygame**

Transformar el juego en una aplicación gráfica con Pygame:

- Crear un menú interactivo con botones.
- Mostrar preguntas, opciones y puntaje de forma visual (según la temática y estructura del juego elegida)
- Controlar eventos (clics, teclas) y posicionar elementos.
- Diseñar la interfaz según configuraciones del archivo JSON.

Se evaluará:

- Uso de superficies, texto, eventos y animaciones básicas.
- Separación entre lógica y presentación.
- Mantenimiento del funcionamiento lógico del juego.

◆ Sprint 6 – Accesibilidad y personalización

El juego debe poder adaptarse a distintas condiciones neurológicas o perceptuales. Cada grupo deberá implementar al menos **una de las siguientes adaptaciones** según el perfil del jugador, configurada desde el archivo JSON.

✚ Adaptaciones posibles:

◆ Autismo (TEA):

- Colores suaves y menos estímulos visuales.
- Flujo de juego siempre visible y estructurado.
- Iconografía o pictogramas junto al texto.
- Indicaciones claras y predecibles.

● Daltonismo:

- Uso de paletas cromáticas seguras.
- Evitar colores como único indicador (agregar texto o símbolos).
- Opción para simular modos: protanopia (déficit en la percepción del rojo), deuteranopia (déficit en la percepción del verde), tritanopia (déficit en la percepción del azul).

● Déficit de Atención (TDA/H):

- Juego rápido y dinámico.
- Feedback inmediato y mensajes motivadores.
- Temporizadores visibles.
- Dividir el juego en bloques cortos o agregar pausas.
- Gamificación (recompensas visuales por aciertos).

Estas opciones deben activarse desde un archivo `config.json`, por ejemplo:

json

```
{  
  "modo": "rapido",  
  "accesibilidad": {  
    "autismo": true,  
    "daltonismo": "protanopia",  
    "tdah": true  
  }  
}
```

Evaluación

El trabajo se evaluará por etapas, considerando:

- Correctitud funcional.
- Uso adecuado de estructuras de datos y archivos.
- Modularidad y estilo de código.
- Originalidad en la interfaz.
- Inclusión real y efectiva de accesibilidad.

Temario:

- **Strings:** manipulación de cadenas mediante algoritmos de desarrollo propio o métodos de la clase str.
- **TDA** (Tipos de datos avanzados: listas, sets, tuplas, diccionarios)
- **Paradigma funcional:** ciudadanos de primera clase, funciones puras, funciones genéricas y reutilizables.
- **Principios DRY.**
- **Lecto/Escritura de archivos** csv, texto y JSON.
- **Biblioteca Pygame:**
 - Configuraciones.
 - Posicionamiento dentro de la pantalla.
 - Manipulación de imágenes.
 - Movimientos en X e Y.
 - Sonidos.
 - Colisiones.
 - Eventos.



Recursos Pygame:

<https://www.youtube.com/playlist?list=PLE9qW09sJEPRFCewXDh1K8Cg4Jdp6LMm>



Recursos Para Accesibilidad:

AUTISMO:

<https://www.who.int/news-room/fact-sheets/detail/autism-spectrum-disorders>

DALTONISMO:

<https://www.nei.nih.gov/learn-about-eye-health/eye-conditions-and-diseases/color-blindness>

PALETAS ACCESIBLES A DISTINTOS TIPOS DE DALTONISMO:

<https://davidmathlogic.com/colorblind/#%23D81B60-%231E88E5-%23FFC107-%23004D40>

TDAH:

<https://chadd.org/about-adhd/overview/>